In [3]:
```python
import sounddevice as sd
import librosa
import numpy as np
from scipy.io.wavfile import write
import pandas as pd
from pydub import AudioSegment
from pydub.silence import split_on_silence
```

In [4]:
```python
# n : nombre de fois que l'utilisateur enregistre l'audio

def record_audio():
    fs = 44100
    duration = 3

    print("how many recordings?? ")
    n = int(input())

    for i in range(0, n+1):
        print("recording started")

        rec = sd.rec(int((duration * fs)), samplerate=fs, channels=1)

        sd.wait()

        print("recording stopped")

        fileName = "C:\\Users\\JUNIOR\\Desktop\\Traitement_signal\\model_\\audio_user1\\" + str(i) + ".wav"
        write(filename=fileName, rate=fs, data=rec)

        print("record again ?\n1 for yes and 0 for no")
        choice = int(input())
        if choice == 0:
            break
```

In [ ]:
```python
def load_and_normalize_audio(file_path, target_db=-20.0):
    # Chargement du fichier audio
    audio, sr = librosa.load(file_path, sr=None)

    # Normalisation du volume audio
    target_rms = 10.0 ** (target_db / 20.0)
    rms = np.sqrt(np.mean(audio**2))
    normalized_audio = audio * (target_rms / rms)

    return normalized_audio, sr
```

In [23]:
```python
# retourne 40 mfccs qui est un array qui contient la moyenne de 40 coef cepstraux
def extract_mfcc(file, n_mfcc=40):
    audio, sr = librosa.load(file)
    mfccs = np.mean(librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=n_mfcc).T, axis=0)
    return mfccs
```

Phrase

1. "Bonjour, comment ça va ?"
2. "La pluie en novembre est abondante."
3. "Quelle est votre couleur préférée ?"
4. "Veuillez répéter la phrase suivante : Authentification vocale."
5. "J'aime écouter de la musique le week-end."
6. "Le soleil brille toujours après la pluie."
7. "Quel est votre plat préféré ?"
8. "Un voyage de mille kilomètres commence par un seul pas."
9. "Répétez après moi : Bonjour, je suis [nom de l'utilisateur]."
10. "Les oiseaux chantent joyeusement le matin."
11. "Avez-vous déjà visité un musée d'art moderne ?"
12. "La vie est une aventure audacieuse ou rien du tout."
13. "Quel temps fait-il aujourd'hui ?"
14. "Répétez cette phrase : Authentification biométrique."
15. "Les chats sont des animaux très indépendants."
16. "Quel est le dernier livre que vous avez lu ?"
17. "Le rire est le meilleur remède."
18. "Cet enregistrement est destiné à l'authentification vocale."
19. "Décrivez votre journée en quelques mots."
20. "Les étoiles brillent dans le ciel nocturne."
21. "Quelle est votre série TV préférée ?"
22. "La musique adoucit les mœurs."
23. "Répétez après moi : Je suis [prénom]."

24. "Le café du matin est essentiel pour beaucoup."
25. "Quel est votre sport préféré ?"
26. "La technologie évolue rapidement."
27. "Les roses sont rouges, les violettes sont bleues."
28. "Qu'est-ce qui vous passionne dans la vie ?"
29. "Répétez cette phrase : Vérification de la voix."
30. "Le silence est parfois la meilleure réponse."
31. "Quel est votre rêve le plus fou ?"
32. "La patience est une vertu."
33. "Décrivez votre endroit préféré sur terre."
34. "Les montagnes sont majestueuses et imposantes."
35. "Répétez après moi : Mon mot de passe vocal."
36. "Quel est le dernier film que vous avez regardé ?"
37. "La créativité est contagieuse, faites-la passer."
38. "Parlez de votre hobby préféré."
39. "Les nuages flottent doucement dans le ciel."
40. "La confiance en soi est la clé du succès."

Ces phrases peuvent varier en complexité et en thème, ce qui permettra d'obtenir une diversité dans les enregistrements vocaux pour l'authentification biométrique.

In [88]:
```
def createMfccCsv():
    df = pd.DataFrame(columns=range(0,40))
    for i in range(0, 64):
        fileName = "C:\\Users\\JUNIOR\\Desktop\\Traitement_signal\\model_\\audio_user1\\" + str(i) + ".wav"
        mfccs = extract_mfcc(fileName, 40)
        lst = list(mfccs)
        df.loc[len(df)] = lst
    df["speaker"] = 1
    df.to_csv("C:\\Users\\JUNIOR\\Desktop\\Traitement_signal\\model_\\audio_user1\\user1.csv",index = False)
    # chemin vers ou enregistrer le fichier csv
```

In [89]:
```
# Ajouter les donnees a complete_data.csv
def appendIndividualToCompleteCSV():
    df1 = pd.read_csv("C:\\Users\\JUNIOR\\Desktop\\Traitement_signal\\model_\\complete_data.csv")
    df2 = pd.read_csv("C:\\Users\\JUNIOR\\Desktop\\Traitement_signal\\model_\\audio_user1\\user1.csv")
    df1 = df1.append(df2, ignore_index=True)
    df1.to_csv("C:\\Users\\JUNIOR\\Desktop\\Traitement_signal\\model_\\complete_data.csv",index = False)
```

In [10]:
```
df = pd.DataFrame(columns=range(0,40))
df
```

Out[10]:

0	1	2	3	4	5	6	7	8	9	...	30	31	32	33	34	35	36	37	38	39

0 rows × 40 columns

In [11]:
```
df["speaker"] = ''
```

In [12]:
```
df
```

Out[12]:

0	1	2	3	4	5	6	7	8	9	...	31	32	33	34	35	36	37	38	39	speaker

0 rows × 41 columns

In [13]:
```
df.to_csv("C:\\Users\\JUNIOR\\Desktop\\Traitement_signal\\model_\\complete_data.csv",index = False) # creation du fic
```

enregistrement des donnees vocaux pour le user_1 (junior)

La qualite n'etant pas bonne, j'ai enregistre avec mon iphone

In [90]:
```
createMfccCsv()
```

In [99]:
```
appendIndividualToCompleteCSV()
```

```
C:\Users\JUNIOR\AppData\Local\Temp\ipykernel_19948\1538524291.py:5: FutureWarning: The frame.append method is depre
cated and will be removed from pandas in a future version. Use pandas.concat instead.
  df1 = df1.append(df2, ignore_index=True)
```

In [91]:
```
df1 = pd.read_csv("C:\\Users\\JUNIOR\\Desktop\\Traitement_signal\\model_\\audio_user1\\user1.csv")
df1
```

Out[91]:

	0	1	2	3	4	5	6	7	8	9	...	31	32	
0	-539.68640	135.369260	29.736536	6.630505	-23.913910	22.557087	-2.331684	-4.480745	-0.051146	-6.453157	...	-3.746106	-1.174224	-1
1	-437.40662	211.010710	17.693949	-5.462568	-2.349320	20.006798	-14.601466	-10.334573	-0.329313	-1.568162	...	0.015173	-4.072109	-1
2	-472.29250	126.657074	22.863264	21.693130	-16.626850	22.664772	-3.265864	-16.270523	-9.668083	-2.283733	...	0.789180	-4.441406	-3
3	-444.08832	146.418350	34.449720	26.976673	-25.371570	28.802416	-10.249216	-12.984854	-5.158410	-0.512292	...	-2.057891	-3.326472	-2
4	-468.77716	145.172240	27.808958	36.665940	-17.847786	24.717510	-8.347945	-14.731977	1.917718	-0.305740	...	-2.046411	-1.352886	-2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	
59	-416.83063	162.092470	19.032839	20.642365	-3.834050	34.474710	-21.536364	-19.733103	-4.123245	3.496007	...	-0.081048	-3.041493	-0
60	-422.75030	160.448790	21.413792	22.238516	-8.212817	24.769365	-16.968170	-25.297482	-0.181527	1.810321	...	-0.269867	-1.324715	-1
61	-423.57635	147.416100	11.257322	38.978573	-0.310137	23.764470	-15.331974	-9.008574	-0.510360	-2.927229	...	4.045573	-2.368570	-0
62	-440.88170	164.966860	13.250927	2.335574	-7.002881	12.450810	-8.744528	-10.620575	-8.622952	5.456815	...	1.390365	-2.200498	0
63	-448.32910	163.466430	15.079639	7.859161	-4.908479	15.847547	-10.682187	-11.764126	-1.453270	-3.591270	...	2.071489	-2.727946	-0

64 rows × 41 columns

In [100]:
```
df1 = pd.read_csv("C:\\Users\\JUNIOR\\Desktop\\Traitement_signal\\model_\\complete_data.csv")
df1
```

Out[100]:

	0	1	2	3	4	5	6	7	8	9	...	31	32	
0	-542.75854	74.108460	11.741064	23.034897	12.914109	18.496325	5.234561	7.675319	1.605489	4.611616	...	-3.376605	-1.186665	-
1	-562.67930	82.183890	13.133499	26.944440	17.091560	20.748460	2.954842	7.096345	1.156457	4.249637	...	-2.034832	-1.293010	-
2	-544.98100	82.920190	7.382197	27.497900	16.678219	16.179867	3.111254	5.963504	0.959272	5.234600	...	-4.010783	-0.966576	-
3	-543.30450	72.189186	11.006381	26.769442	15.081501	15.482657	5.437983	6.266879	0.044376	5.718420	...	-3.440320	-1.326851	-
4	-550.72360	91.334694	11.105228	25.450626	14.965750	20.898336	3.806478	6.497824	0.432799	2.931146	...	-2.849905	-1.164759	-
...	...	...	...	...	...	...	...	...	...	...	...	...	...	
157	-416.83063	162.092470	19.032839	20.642365	-3.834050	34.474710	-21.536364	-19.733103	-4.123245	3.496007	...	-0.081048	-3.041493	-
158	-422.75030	160.448790	21.413792	22.238516	-8.212817	24.769365	-16.968170	-25.297482	-0.181527	1.810321	...	-0.269867	-1.324715	-
159	-423.57635	147.416100	11.257322	38.978573	-0.310137	23.764470	-15.331974	-9.008574	-0.510360	-2.927229	...	4.045573	-2.368570	-
160	-440.88170	164.966860	13.250927	2.335574	-7.002881	12.450810	-8.744528	-10.620575	-8.622952	5.456815	...	1.390365	-2.200498	
161	-448.32910	163.466430	15.079639	7.859161	-4.908479	15.847547	-10.682187	-11.764126	-1.453270	-3.591270	...	2.071489	-2.727946	-

162 rows × 41 columns

In [98]:
```
df1.to_csv("C:\\Users\\JUNIOR\\Desktop\\Traitement_signal\\model_\\complete_data.csv",index = False)
```

speaker 1 : michel

speaker 2 : diop

In [101]:
```
import pandas as pd
from sklearn.neural_network import MLPClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.naive_bayes import MultinomialNB
from sklearn.naive_bayes import ComplementNB
from sklearn.naive_bayes import BernoulliNB
from sklearn import svm
from sklearn.model_selection import train_test_split
import sounddevice as sd
from scipy.io.wavfile import write
from sklearn.metrics import confusion_matrix,precision_score
import time
```

In [108]:
```
def predictor():
    df = pd.read_csv("C:\\Users\\JUNIOR\\Desktop\\Traitement_signal\\model_\\complete_data.csv")

    Y = df["speaker"]

    X = df.drop(columns=["speaker",])

    print(X.shape, "\n")

    # scaling pending

    X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.15, random_state=1)

    # MLPClassifier
    classifier = MLPClassifier(solver='adam', alpha=0.001,
                               random_state=1, max_iter=500,
                               hidden_layer_sizes=100, activation="logistic")
    classifier.fit(X_train, Y_train)
    pred_mlp = classifier.predict(X_test)
    cm_mlp = confusion_matrix(Y_test, pred_mlp)
    # Calcul de la précision
    #precision = precision_score(Y_test, pred_mlp)
    # Affichage de la précision
    #print("Précision du modèle MLPclassifier :", precision)
    print("Score of MLPClassifier: ",classifier.score(X_test, Y_test))
    print("cm of mlp: \n", cm_mlp)

    # decision tree classifier
    clf = DecisionTreeClassifier(random_state=1)
    clf.fit(X_train, Y_train)
    pred_tree = clf.predict(X_test)
    cm_tree = confusion_matrix(Y_test, pred_tree)
    # Calcul de la précision
    #precision = precision_score(Y_test, pred_tree)
    # Affichage de la précision
    #print("Précision du modèle tree classifier :", precision)
    print("Score for decision tree classifier: ", clf.score(X_test,Y_test))
    print("cm of tree: \n", cm_tree)
    print(pred_tree, "\n")
    print(Y_test)

    # random forest classifier
    clf_forest = RandomForestClassifier(max_depth=100, random_state=1
                                        , n_estimators=8, criterion="entropy",
                                        max_features="auto")
    clf_forest.fit(X_train, Y_train)
    pred_forest = clf_forest.predict(X_test)
    cm_forest = confusion_matrix(Y_test, pred_forest)
    # Calcul de la précision
    #precision = precision_score(Y_test, pred_forest)
    # Affichage de la précision
    #print("Précision du modèle forest classifier :", precision)
    print("Score for Random forest classifier: ", clf_forest.score(X_test, Y_test))
    print("cm of forest: \n", cm_forest)

    # K nearest neighbours classifier
    neigh = KNeighborsClassifier(n_neighbors=5, weights="uniform"
                                 , algorithm="ball_tree")
    neigh.fit(X_train, Y_train)
    pred_neigh = neigh.predict(X_test)
    cm_neigh = confusion_matrix(Y_test, pred_neigh)
    # Calcul de la précision
    #precision = precision_score(Y_test, pred_neigh)
    # Affichage de la précision
    #print("Précision du modèle neighbours classifier :", precision)
    print("Score for KNN classifier: ", neigh.score(X_test, Y_test))
    print("cm of K neighbour: \n", cm_neigh)

    # Naive Bayes gaussian
    gnb = GaussianNB()
    gnb.fit(X_train, Y_train)
    pred_gnb = gnb.predict(X_test)
    cm_gnb = confusion_matrix(Y_test, pred_gnb)
    # Calcul de la précision
    #precision = precision_score(Y_test, pred_gnb)
    # Affichage de la précision
    #print("Précision du modèle Naive gaussian classifier :", precision)
    print("Score for gaussian NB classifier: ", gnb.score(X_test,Y_test))
    print("cm of gnb: \n", cm_gnb)

    # Naive bayes bernoulli
    bnb = BernoulliNB()
    bnb.fit(X_train, Y_train)
    pred_bnb = bnb.predict(X_test)
    cm_bnb = confusion_matrix(Y_test, pred_bnb)
    # Calcul de la précision
    #precision = precision_score(Y_test, pred_bnb)
    # Affichage de la précision
```

```
    #print("Précision du modèle  naive bernoulli classifier :", precision)
    print("Score for bernoulli NB classifier: ", bnb.score(X_test,Y_test))
    print("cm of bnb: \n", cm_bnb)

    #SVM
    svmc = svm.SVC()
    svmc.fit(X_train, Y_train)
    pred_svm = svmc.predict(X_test)
    cm_svm = confusion_matrix(Y_test, pred_svm)
    print("Score for SVM classifier: ", svmc.score(X_test,Y_test))
    # Calcul de la précision
    #precision = precision_score(Y_test, pred_svm)
    # Affichage de la précision
    #print("Précision du modèle svm classifier :", precision)
    print("cm of svm: \n", cm_svm)
```

In [109]:
```
predictor()
```

```
(162, 40)

Score of MLPClassifier:  1.0
cm of mlp:
 [[15  0]
 [ 0 10]]
Score for decision tree classifier:  1.0
cm of tree:
 [[15  0]
 [ 0 10]]
[0 0 1 0 1 0 1 0 1 0 0 1 0 0 1 0 1 0 0 1 1 0 0 1 0]

44     0
47     0
110    1
77     0
159    1
11     0
116    1
35     0
143    1
14     0
78     0
99     1
91     0
19     0
138    1
75     0
107    1
81     0
85     0
160    1
102    1
29     0
39     0
123    1
95     0
Name: speaker, dtype: int64
Score for Random forest classifier:  1.0
cm of forest:
 [[15  0]
 [ 0 10]]
Score for KNN classifier:  1.0
cm of K neighbour:
 [[15  0]
 [ 0 10]]
Score for gaussian NB classifier:  1.0
cm of gnb:
 [[15  0]
 [ 0 10]]
Score for bernoulli NB classifier:  1.0
cm of bnb:
 [[15  0]
 [ 0 10]]
Score for SVM classifier:  1.0
cm of svm:
 [[15  0]
 [ 0 10]]

C:\Users\JUNIOR\AppData\Local\anaconda3\lib\site-packages\sklearn\ensemble\_forest.py:424: FutureWarning: `max_feat
ures='auto'` has been deprecated in 1.1 and will be removed in 1.3. To keep the past behaviour, explicitly set `max
_features='sqrt'` or remove this parameter as it is also the default value for RandomForestClassifiers and ExtraTre
esClassifiers.
  warn(
```

Nous observons un score de precision de 1 pour certain modele

In [110]:
```
def verifyUser():
    fs = 44100
    duration = 3

    print("Prepare toi a parler...")

    # wait = sd.rec(5, samplerate=fs, channels=1)
    # sd.wait()
    time.sleep(3)
    print("recording started")

    rec = sd.rec(int((duration * fs)), samplerate=fs, channels=1)

    sd.wait()

    print("recording stopped")

    file = "C:\\Users\\JUNIOR\\Desktop\\Traitement_signal\\model_\\last_try.wav"
    write(filename=file, rate=fs, data=rec)

    mfcc = extract_mfcc(file, n_mfcc=40)

    input = pd.DataFrame(columns=range(0, 40))

    lst = list(mfcc)
    input.loc[len(input)] = lst

    df = pd.read_csv("C:\\Users\\JUNIOR\\Desktop\\Traitement_signal\\model_\\complete_data.csv")

    Y = df["speaker"]
    X = df.drop(columns=["speaker"])

    # MLPClassifier
    classifier = MLPClassifier(solver='adam', alpha=0.001,
                               random_state=1, max_iter=500,
                               hidden_layer_sizes=100, activation="logistic")
    classifier.fit(X, Y)
    pred_mlp = classifier.predict(input)
    print("the MLP CLASSIFIER speaker is ", pred_mlp)

    # decision tree

    clf = DecisionTreeClassifier(random_state=1)
    clf.fit(X, Y)
    pred_tree = clf.predict(input)
    print("the DECISION TREE speaker is ", pred_tree)

    # random forest
    clf_forest = RandomForestClassifier(max_depth=100, random_state=1
                                        , n_estimators=75, criterion="entropy",
                                        max_features="auto")
    clf_forest.fit(X, Y)
    pred_rf = clf_forest.predict(input)
    print("the RANDOM FOREST speaker is: ", pred_rf)

    # K nearest neighbours classifier
    neigh = KNeighborsClassifier(n_neighbors=3)
    neigh.fit(X, Y)
    pred_knn = neigh.predict(input)
    print("the KNN speaker is : ", pred_knn)

    # Naive Bayes gaussian
    gnb = GaussianNB()
    gnb.fit(X, Y)
    pred_gnb = gnb.predict(input)
    print("the gaussian NB speaker is : ", pred_gnb)

    # Naive bayes bernoulli
    bnb = BernoulliNB()
    bnb.fit(X, Y)
    pred_bnb = bnb.predict(input)
    print("The bernoulli NB speaker is: ", pred_bnb)
```

In [117]:
```
verifyUser()
```

```
Prepare toi a parler...
recording started
recording stopped
the MLP CLASSIFIER speaker is  [0]
the DECISION TREE speaker is  [0]
the RANDOM FOREST speaker is:  [0]
the KNN speaker is :  [0]
the gaussian NB speaker is :  [0]
The bernoulli NB speaker is:  [0]

C:\Users\JUNIOR\AppData\Local\anaconda3\lib\site-packages\sklearn\base.py:420: UserWarning: X does not have valid f
eature names, but MLPClassifier was fitted with feature names
  warnings.warn(
C:\Users\JUNIOR\AppData\Local\anaconda3\lib\site-packages\sklearn\base.py:420: UserWarning: X does not have valid f
eature names, but DecisionTreeClassifier was fitted with feature names
  warnings.warn(
C:\Users\JUNIOR\AppData\Local\anaconda3\lib\site-packages\sklearn\ensemble\_forest.py:424: FutureWarning: `max_feat
ures='auto'` has been deprecated in 1.1 and will be removed in 1.3. To keep the past behaviour, explicitly set `max
_features='sqrt'` or remove this parameter as it is also the default value for RandomForestClassifiers and ExtraTre
esClassifiers.
  warn(
C:\Users\JUNIOR\AppData\Local\anaconda3\lib\site-packages\sklearn\base.py:420: UserWarning: X does not have valid f
eature names, but RandomForestClassifier was fitted with feature names
  warnings.warn(
C:\Users\JUNIOR\AppData\Local\anaconda3\lib\site-packages\sklearn\base.py:420: UserWarning: X does not have valid f
eature names, but KNeighborsClassifier was fitted with feature names
  warnings.warn(
C:\Users\JUNIOR\AppData\Local\anaconda3\lib\site-packages\sklearn\base.py:420: UserWarning: X does not have valid f
eature names, but GaussianNB was fitted with feature names
  warnings.warn(
C:\Users\JUNIOR\AppData\Local\anaconda3\lib\site-packages\sklearn\base.py:420: UserWarning: X does not have valid f
eature names, but BernoulliNB was fitted with feature names
  warnings.warn(
```

In [114]:
```
def verifyUser2():
    fs = 44100
    duration = 3

    print("Prepare toi a parler...")

    # wait = sd.rec(5, samplerate=fs, channels=1)
    # sd.wait()
    time.sleep(3)
    print("recording started")

    rec = sd.rec(int((duration * fs)), samplerate=fs, channels=1)

    sd.wait()

    print("recording stopped")

    file = "C:\\Users\\JUNIOR\\Desktop\\Traitement_signal\\model_\\last_try.wav"
    write(filename=file, rate=fs, data=rec)

    mfcc = extract_mfcc(file, n_mfcc=40)

    input = pd.DataFrame(columns=range(0, 40))

    lst = list(mfcc)
    input.loc[len(input)] = lst

    df = pd.read_csv("C:\\Users\\JUNIOR\\Desktop\\Traitement_signal\\model_\\complete_data.csv")

    Y = df["speaker"]
    X = df.drop(columns=["speaker"])

    # MLPClassifier
    classifier = MLPClassifier(solver='adam', alpha=0.001,
                               random_state=1, max_iter=500,
                               hidden_layer_sizes=100, activation="logistic")
    classifier.fit(X, Y)
    pred_mlp = classifier.predict_proba(input)
    p1 = classifier.predict(input)
    print(f"the MLP CLASSIFIER speaker is {p1}\n", pred_mlp[0])

    # decision tree

    clf = DecisionTreeClassifier(random_state=1)
    clf.fit(X, Y)
    pred_tree = clf.predict_proba(input)
    p2 = clf.predict(input)
    print(f"the DECISION TREE speaker is {p2} \n", pred_tree[0][0])

    # random forest
    clf_forest = RandomForestClassifier(max_depth=100, random_state=1
                                        , n_estimators=75, criterion="entropy",
                                        max_features="auto")
    clf_forest.fit(X, Y)
    pred_rf = clf_forest.predict_proba(input)
    p3 = clf_forest.predict(input)
    print(f"the RANDOM FOREST speaker is: {p3} \n", pred_rf[0][0])

    # K nearest neighbours classifier
    neigh = KNeighborsClassifier(n_neighbors=3)
    neigh.fit(X, Y)
    pred_knn = neigh.predict_proba(input)
    p4 = neigh.predict(input)
    print(f"the KNN speaker is {p4} :\n ", pred_knn[0][0])

    # Naive Bayes gaussian
    gnb = GaussianNB()
    gnb.fit(X, Y)
    pred_gnb = gnb.predict_proba(input)
    p5 = gnb.predict(input)
    print(f"the gaussian NB speaker is {p5} \n: ", pred_gnb[0][0])

    # Naive bayes bernoulli
    bnb = BernoulliNB()
    bnb.fit(X, Y)
    pred_bnb = bnb.predict_proba(input)
    p6 = bnb.predict(input)
    print(f"The bernoulli NB speaker is: {p6} \n", pred_bnb)
```

In [115]:
```
verifyUser2()
```

```
Prepare toi a parler...
recording started
recording stopped
the MLP CLASSIFIER speaker is [0]
 [0.98877648 0.01122352]
the DECISION TREE speaker is [0]
 1.0
the RANDOM FOREST speaker is: [0]
 0.72
the KNN speaker is [0] :
  0.6666666666666666
the gaussian NB speaker is [0]
:  0.9999999999997442
The bernoulli NB speaker is: [0]
 [[0.99785711 0.00214289]]

C:\Users\JUNIOR\AppData\Local\anaconda3\lib\site-packages\sklearn\base.py:420: UserWarning: X does not have valid f
eature names, but MLPClassifier was fitted with feature names
  warnings.warn(
C:\Users\JUNIOR\AppData\Local\anaconda3\lib\site-packages\sklearn\base.py:420: UserWarning: X does not have valid f
eature names, but MLPClassifier was fitted with feature names
  warnings.warn(
C:\Users\JUNIOR\AppData\Local\anaconda3\lib\site-packages\sklearn\base.py:420: UserWarning: X does not have valid f
eature names, but DecisionTreeClassifier was fitted with feature names
  warnings.warn(
C:\Users\JUNIOR\AppData\Local\anaconda3\lib\site-packages\sklearn\base.py:420: UserWarning: X does not have valid f
eature names, but DecisionTreeClassifier was fitted with feature names
  warnings.warn(
C:\Users\JUNIOR\AppData\Local\anaconda3\lib\site-packages\sklearn\ensemble\_forest.py:424: FutureWarning: `max_feat
ures='auto'` has been deprecated in 1.1 and will be removed in 1.3. To keep the past behaviour, explicitly set `max
_features='sqrt'` or remove this parameter as it is also the default value for RandomForestClassifiers and ExtraTre
esClassifiers.
  warn(
C:\Users\JUNIOR\AppData\Local\anaconda3\lib\site-packages\sklearn\base.py:420: UserWarning: X does not have valid f
eature names, but RandomForestClassifier was fitted with feature names
  warnings.warn(
C:\Users\JUNIOR\AppData\Local\anaconda3\lib\site-packages\sklearn\base.py:420: UserWarning: X does not have valid f
eature names, but RandomForestClassifier was fitted with feature names
  warnings.warn(
C:\Users\JUNIOR\AppData\Local\anaconda3\lib\site-packages\sklearn\base.py:420: UserWarning: X does not have valid f
eature names, but KNeighborsClassifier was fitted with feature names
  warnings.warn(
C:\Users\JUNIOR\AppData\Local\anaconda3\lib\site-packages\sklearn\base.py:420: UserWarning: X does not have valid f
eature names, but KNeighborsClassifier was fitted with feature names
  warnings.warn(
C:\Users\JUNIOR\AppData\Local\anaconda3\lib\site-packages\sklearn\base.py:420: UserWarning: X does not have valid f
eature names, but GaussianNB was fitted with feature names
  warnings.warn(
C:\Users\JUNIOR\AppData\Local\anaconda3\lib\site-packages\sklearn\base.py:420: UserWarning: X does not have valid f
eature names, but GaussianNB was fitted with feature names
  warnings.warn(
C:\Users\JUNIOR\AppData\Local\anaconda3\lib\site-packages\sklearn\base.py:420: UserWarning: X does not have valid f
eature names, but BernoulliNB was fitted with feature names
  warnings.warn(
C:\Users\JUNIOR\AppData\Local\anaconda3\lib\site-packages\sklearn\base.py:420: UserWarning: X does not have valid f
eature names, but BernoulliNB was fitted with feature names
  warnings.warn(
```